Your customers subscribe. You never learn who they are.

Styx is a shielded payment and subscription protocol on Solana. The merchant gets paid on a schedule they set. No name, no email, no card number, no account. Nothing to store, nothing to lose.

SOLANA DEVNET · NOT AUDITED · NO MAINNET DEPLOYMENT

Every subscription asks for four things it does not need.

A name. An email. A card number. And a permanent record that you are a customer of this particular thing. The merchant rarely wanted any of it. The processor required it, the database kept it, and the compliance team inherited it.

That record is a liability on the merchant's balance sheet and an exposure on the customer's. Both sides would drop it if there were a way to still get paid.

Recurring revenue is the model the whole software industry runs on. It is also the model that forces the deepest customer file. Styx separates the two: the money recurs, the file never exists.

Four steps, and the merchant is only in the first one.

01

Pay

The customer pays through whatever checkout the merchant already runs. That side does not change, and the merchant keeps their existing settlement.

02

Verify

On settlement, one call mints an access code: 32 bytes from a cryptographic random source. It does not expire. Someone who has paid can come back and use it whenever they want.

03

Redeem

The code is spent exactly once. Redemption is an atomic increment, so two people racing on the same code receive one entitlement between them, never two.

04

Subscribe

A one time account is created and funded. It pays the merchant a fixed amount per period, for the periods that were bought, and then it closes.

> The merchant is paid at step one and paid again every period. They are never handed a customer record, because at no point in this sequence is one created.

What a merchant on Styx never receives.

- No name
- No email address
- No card number and no card processor
- No billing address
- No account, and therefore no password
- No customer database to breach

WHAT THEY DO RECEIVE

The money. A fixed amount per period, at an interval they set, held on chain and claimable by them and nobody else. Entitlement they can check against the chain instead of against a support ticket.

A merchant who works with us is not making a sacrifice. They are removing the single most expensive thing on their compliance surface, and keeping the revenue.

Data you never collected cannot leak, cannot be subpoenaed, and cannot be sold by someone you later acquire.

THE MECHANISM

The subscription account is addressed by a secret, not by a wallet.

A normal on chain subscription is a public statement that a given wallet pays a given merchant. Ours is not: the account is derived from a commitment to a note secret. There is no address to rederive from a wallet, so there is no way to ask the chain whether a particular person subscribes to a particular service. **With one exception, and it is on chain today:** two of the twenty-eight live vaults are legacy normal-mode and carry the subscriber's wallet in the clear, so for those two the question can be asked directly. Counted 26 August; one of the two is our own deployment wallet.

ACCESS CODE

no expiry

Single use, enforced by an atomic increment. Redeem it whenever.

SUBSCRIPTION ACCOUNT

1

One time, funded once, closes when its last period is claimed.

AMOUNT AND INTERVAL

set

Fixed per period, in slots, chosen by the merchant and public.

EXECUTED ON DEVNET, 22 AUGUST 2026

The buyer's address is not in the transaction.

This is not a design intention. It is a transaction anyone can open. The customer signs one ordinary payment to the deployment. The deployment funds a single use key from a different address of its own. That key makes the deposit.

STEP	SIGNATURE	EFFECT
payment	`3gGjM7TZCDPk…`	one customer signature
funding	`jxXoRhKbqsRG…`	the deployment funds a single use key
deposit	`27KkoTi1hg6U…`	leaf 72, slot 486 353 558

> Re-read from the chain rather than believed from the app: the deposit succeeded, its fee payer is the single use key, and **the customer's address does not appear anywhere in its account keys.**

LIVE, END TO END

Purchase to running subscription in 167 seconds.

One command, on devnet, frozen on a tagged branch so the run you watch on 4 September is the run that is recorded here.

ARTEFACT	VALUE	
deposit	`5DiMyNkJdZxa…`	`leaf 33`
subscription	`4E39AJ37BFZq…`	`SubscribePrivateStark`
compute used	`36 127 of 200 000 CU`	
merchant vault	`7xUisNg8HKhg…`	`1.003 403 44 SOL`

> Every line comes from `solana confirm`, not from the app that sent it. A client that believes it sent something is not evidence that it landed.

Post-quantum by construction, and the verifier says no.

Hash based STARK proofs. No elliptic curves anywhere in the proof, so there is no curve for a quantum computer to break. No trusted setup, so there is no ceremony anyone has to believe in and no toxic waste to have been destroyed.

The proof is checked by a program on Solana, not by a server we run. All three outcomes were measured against it on chain.

HONEST PROOF

accepted

809 812 compute units.

FORGED BY ONE BYTE

rejected

542 150 units, deep in the proof work.

TAMPERED INPUT

rejected

19 777 units, immediately.

WHAT IS RUNNING TODAY

Ten programs live on chain. 4 216 tests green.

PROGRAMS LIVE
10
executable at their declared address, checked at slot 486 742 009.

TESTS PASSING
4 216
across 16 packages and the Rust workspace, zero failures.

NOTES IN THE POOL
47
unspent right now, of 73 deposited.

COST PER PURCHASE
0.0005
SOL of network fees for a full private journey.

Every deposit lands in one denomination on purpose. A crowd does not add across denominations, it splits, so the same capital spread over six pools buys six small crowds instead of one that works.

Which is why the honest number today is one, not forty seven. We do not encrypt the withdrawal, we make it indistinguishable from the others, so the guarantee is a crowd and it is worth nothing at a crowd of one. Four of our five spends still republish the commitment their deposit already published, and any one channel that partitions the pool — the fee payer, the deposit funder, timing — pins the crowd to one however many notes the tree holds. C7 closed that channel for the withdrawal on 25 August, and one real spend has landed with no commitment on the wire. The rest is the work, and it is the only asset here that grows for free once it starts.

One percent, and the crowd grows with every merchant.

THE REVENUE

One percent of the purchase, taken in the customer's own transaction, at the moment of sale. No subscription fee for the merchant, no integration invoice, no seat pricing. We are paid when they are paid.

THE COMPOUNDING PART

Every merchant shares the same pool per denomination. A new partner does not get their own private corner: they make every existing customer harder to single out, and every existing customer makes theirs harder too.

Privacy protocols usually get better with volume and worse with fragmentation. We built the economics so that signing a partner does both jobs at once.

Three things, in the order they matter.

VERIFY
01

Run the demo yourself. One command, and every transaction on these plates is public. Do not take the deck's word for any of it.

USE
02

Be a merchant on the pool. It costs one percent and no integration rewrite, and it is the fastest way to make the protocol worth more to everyone on it.

BACK
03

Fund the next circuit. It is the piece that takes the guarantee from strong to complete, and it is the work we want to be doing next.